COMPUTER ENGINEERING

QUEZON CITY

| | | | |
|---|---|---|---|
| **COURSE CODE/TITLE:** | CPE 010 | **SECTION:** | CPE21S2 |
| **DATE PERFORMED:** | 10/21/2026 | **DATE SUBMITTED:** | 10/21/2026 |
| **NAME:** | Lara,Juan Paulo C.<br>Mencidor, Renverlanz D. | **INSTRUCTOR:** | Engr. Jimlord Quejado |

## ACTIVITY NO. 14 ALGORITHM COMPLEXITY

### 1. PROCEDURE OUTPUT

What procedures did you perform?

**OUTPUTS**

| | Best Case | Worst Case | Analysis |
|---|---|---|---|
| **Bubble Sort** | O(n), when already sorted with a swapped flag check | O(n^2), when sorted in reverse order | (LARA)<br>This algorithm sorts the elements by comparing each adjacent element and swapping them until the condition is satisfied and the array is sorted. It has a best case of O(n) when the array is already sorted and flagged. It has a worst case of $O(n^2)$, which means as elements increase from 100 to 10,000, it takes longer to sort elements at these numbers.<br><br>(MENCIDOR)<br>Bubble Sort works by repeatedly swapping adjacent elements. In the worst case, it performs $\frac{n(n-1)}{2}$ comparisons and swaps. As n grows from 100 to 10,000 operations scale quadratically, making it inefficient for large inputs. |
| **Merge Sort** | O(n log n) | O(n log n) | (LARA)<br>This algorithm sorts the elements by utilizing a divide-and-conquer scheme, where it divides the array into sub-arrays made of halves. Then, it recursively sorts each element, and merges the sorted ones back into one |

| | | | |
|---|---|---|---|
| | | | complete array. Since this algorithm consistently divides, sorts, and merges these elements, its best and worst case time complexities remain as O(n log n).<br><br>(MENCIDOR)<br>Merge Sort Divides the array into halves, recursively sorts them, and merges the sorted halves. Because the division tree depth is always $log_2 n$ and merging takes O(n) time, both the best and worst cases remain O(n log n) regardless of initial order. |

Table 14-1. Best- and Worst-Case Analysis using Theoretical Tools

| | **Bubble Sort** | **Merge Sort** |
|---|---|---|
| **Best & Worst Case** | Bubble Sort<br>N = 100<br>Best Case: 0.0006 ms<br>Worst Case: 0.0557 ms<br>N = 1000<br>Best Case: 0.0028 ms<br>Worst Case: 4.7087 ms<br>N = 10000<br>Best Case: 0.0264 ms<br>Worst Case: 476.308 ms | Merge Sort<br>N = 100<br>Best Case: 0.1334 ms<br>Worst Case: 0.1286 ms<br>N = 1000<br>Best Case: 1.321 ms<br>Worst Case: 1.3203 ms<br>N = 10000<br>Best Case: 15.6075 ms<br>Worst Case: 17.5284 ms |

**Analysis**

(Lara)

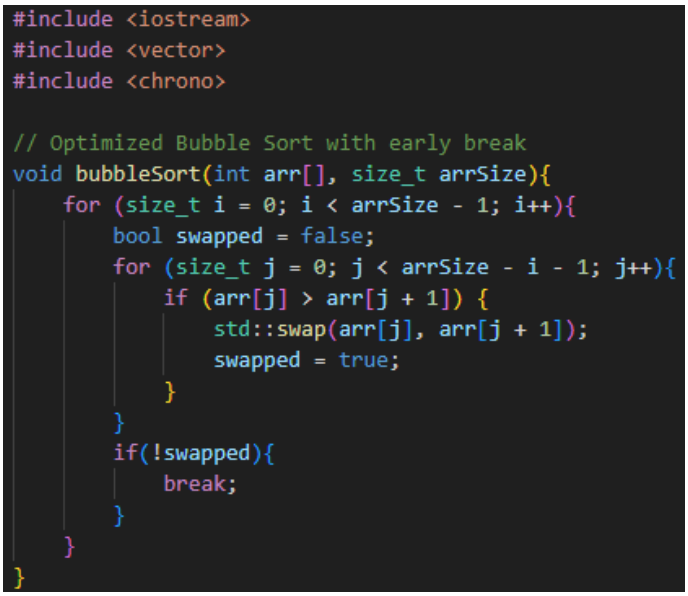
As shown in the console output showing the time for each algorithm to sort an array in best and worst case scenarios, the bubble sort algorithm achieves its O(n) complexity with the flag that stops the algorithm after satisfying the condition. Its time increases linearly as n grows from 100 to 1000, and then 10000. Its worst case scenario shows a quadratic O($n^2$) increase from n number 1000 to 10000, where the execution time increases by 100 times due to the large growth in the number. Merge sort shows a consistent execution time for both case scenarios with an O(n log n) time complexity. The numbers show relatively similar numbers in both scenarios, which demonstrate the consistency of this algorithm. Comparing the two algorithms, bubble sort is faster in smaller numbers of elements but significantly slower than merge sort, where it has a slower growth in execution time..

(Mencidor)

Based on the experimental data and code, optimized Bubble Sort uses an early break flag to achieve a linear O(n) best case time complexity, outperforming Merge Sort on already sorted arrays. However, for reverse

sorted arrays, Bubble Sort devolves into its quadratic O($n^2$) worst case behavior, causing runtime to surge to 476.308 ms at N = 10000. In contrast, Merge Sort demonstrates highly consistent O(n log n) performance across both best and worst cases, taking roughly 15.6 ms to 17.5 ms at N = 10000, because its divide and conquer strategy and memory allocations execute regardless of initial element ordering. While Merge Sort incurs higher recursion and allocation overhead on small inputs like N = 100, its superior asymptotic complexity makes it over 27 times faster than Bubble Sort in the worst case for larger datasets.

Table 14-2. Best- and Worst-Case Analysis using Experimental Tools

| | |
|---|---|
| Bubble Sort |  |

```cpp
#include <iostream>
#include <vector>
#include <chrono>

// Optimized Bubble Sort with early break
void bubbleSort(int arr[], size_t arrSize){
    for (size_t i = 0; i < arrSize - 1; i++){
        bool swapped = false;
        for (size_t j = 0; j < arrSize - i - 1; j++){
            if (arr[j] > arr[j + 1]) {
                std::swap(arr[j], arr[j + 1]);
                swapped = true;
            }
        }
        if(!swapped){
            break;
        }
    }
}
```

```cpp
void testBubbleSort(int n){
    // Already sorted array
    std::vector<int> bestArr(n);
    for (int i = 0; i < n; i++){
        bestArr[i] = i;
    }
    // Reverse sorted array
    std::vector<int> worstArr(n);
    for (int i = 0; i < n; i++){
        worstArr[i] = n - i;
    }
    // Best Case
    auto startBest = std::chrono::high_resolution_clock::now();
    bubbleSort(bestArr.data(), n);
    auto endBest = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> bestTime = endBest - startBest;

    // Worst Case
    auto startWorst = std::chrono::high_resolution_clock::now();
    bubbleSort(worstArr.data(), n);
    auto endWorst = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> worstTime = endWorst - startWorst;

    std::cout << "N = " << n << "\n";
    std::cout << "Best Case: " << bestTime.count() << " ms\n";
    std::cout << "Worst Case: " << worstTime.count() << " ms\n";
}
```

```cpp
int main(){
    std::cout << "Bubble Sort\n";
    testBubbleSort(100);
    testBubbleSort(1000);
    testBubbleSort(10000);
    return 0;
}
```

Merge Sort

```cpp
#include <iostream>
#include <vector>
#include <chrono>

void merge(int arr[], int left, int middle, int right){
    int n1 = middle - left + 1;
    int n2 = right - middle;

    std::vector<int> L(n1);
    std::vector<int> R(n2);

    for (int i = 0; i < n1; i++){
        L[i] = arr[left + i];
    }
    for (int j = 0; j < n2; j++){
        R[j] = arr[middle + 1 + j];
    }
    int i = 0, j = 0, k = left;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) {
            arr[k++] = L[i++];
        } else {
            arr[k++] = R[j++];
        }
    }
    while (i < n1){
        arr[k++] = L[i++];
    }
    while (j < n2){
        arr[k++] = R[j++];
    }
}
```

```cpp
void testMergeSort(int n){
    // Already sorted array
    std::vector<int> bestArr(n);
    for (int i = 0; i < n; i++){
        bestArr[i] = i;
    }
    // Reverse sorted array
    std::vector<int> worstArr(n);
    for (int i = 0; i < n; i++){
        worstArr[i] = n - i;
    }
    // Best Case
    auto startBest = std::chrono::high_resolution_clock::now();
    mergeSort(bestArr.data(), 0, n - 1);
    auto endBest = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> bestTime = endBest - startBest;

    // Worst Case
    auto startWorst = std::chrono::high_resolution_clock::now();
    mergeSort(worstArr.data(), 0, n - 1);
    auto endWorst = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> worstTime = endWorst - startWorst;

    std::cout << "N = " << n << "\n";
    std::cout << "Best Case: " << bestTime.count() << " ms\n";
    std::cout << "Worst Case: " << worstTime.count() << " ms\n";
}

int main(){
    std::cout << "Merge Sort\n";
    testMergeSort(100);
    testMergeSort(1000);
    testMergeSort(10000);
    return 0;
}
```

Table 14-3. Codes Used in Best- and Worst-Case Analysis using Experimental Tools

## 2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

**ILO A**
**Searching Technique 1: Linear**

| | |
|---|---|
| Pseudocode | FUNCTION LinearSearch(array, target):<br>FOR i FROM 0 TO length(array) - 1:<br>IF array[i] == target:<br>RETURN i<br>RETURN -1 |
| Analysis | (Lara)<br>This linear searching algorithm traverses through the list by sequentially reading each element for each iteration until it matches a target element or finds the end of the array. Its best case scenario is O(1) when the matching element is the first element of the array. The |

| | |
|---|---|
| | worst case scenario is O(n) based on the amount of times it iterated through the list. This is usually applied when the element to find is at the last place of the array. Its space complexity is O(1) since it only requires an index for the iterations.<br><br>(Mencidor)<br>The algorithm iterates through every element sequentially until it finds the target or reaches the end. The Best case for the Time Complexity is O(1) where the target is at index 0, while the Worst Case is O(n) where the target is at the last position or not present. Lastly, the Space Complexity is O(1), where it only requires a loop variable index. |

**Searching Technique 2: Binary**

| | |
|---|---|
| Pseudocode | FUNCTION BinarySearch(array, target):<br>low = 0<br>high = length(array) - 1<br>WHILE low <= high:<br>mid = low + (high - low) / 2<br>IF array[mid] == target:<br>RETURN mid<br>ELSE IF array[mid] < target:<br>low = mid + 1<br>ELSE:<br>high = mid - 1<br>RETURN -1 |
| Analysis | (Lara)<br>Compared to the previous algorithm, this requires a sorted array to function correctly. This binary searching algorithm divides the list in half by comparing the middle element with the target value. Its best case scenario is O(1) if the middle value is the value used to find, while the worst case scenario is $O(\log_2 N)$ since it performs the searching operation by halving the parts for each iteration. Its space complexity is O(1) since it uses iteration to perform its search.<br><br>(Mencidor)<br>In this algorithm, it assumes a sorted array. Where it continuously divides the search interval in half by comparing the middle element with the target. While the Best Case for Time Complexity is O(1) where the middle element is the target, the Worst case on the other hand is $O(log_2 n)$ where it halves the search space at each step. Lastly, the Space Complexity is also O(1) because of the iterative approach. |

**Time and Space Complexities**

| Complexity | Technique 1: | Technique 2: |
|---|---|---|
| Time | Best: O(1)<br>Worst: O(N) | Best: O(1)<br>Worst: $O(\log_2 N)$ |

| | | |
|---|---|---|
| Space | O(1) | O(1) |

**Empirical analysis**

| Input Size (N) | Elapsed Time for Binary Search (Worst-Case) | Elapsed Time for Linear Search (Worst-Case) |
|---|---|---|
| 1,000<br>10,000<br>100,000 | Binary Search<br>N = 1000<br>Worst Case: 0.0005 ms<br>N = 10000<br>Worst Case: 0.0002 ms<br>N = 100000<br>Worst Case: 0.0003 ms | Linear Search<br>N = 1000<br>Worst Case: 0.0052 ms<br>N = 10000<br>Worst Case: 0.0497 ms<br>N = 100000<br>Worst Case: 0.4982 ms |

**Analysis**

(Lara)

The empirical data highlights the operational differences between linear scans and logarithmic searches. By testing with missing target values, both functions were tested in the worst case scenarios. While Linear Search must check every single element sequentially, leading to runtime scaling directly with array size. Binary Search rapidly narrows down the search space. The test results showcase why Binary Search is efficient in finding an element in sorted data, which maintains sub millisecond speeds even as the dataset scales up to 100,000 elements.

(Mencidor)

Both algorithms were benchmarked under worst case scenarios using target values absent from the data array (target = -1). While Linear Search checks every single element sequentially, Binary Search eliminates half of the remaining elements at each step. The benchmark results clearly illustrate why logarithmic algorithms are essential for high performance software. While Linear Search runtime scales noticeably with larger data structures, Binary Search runtime remains virtually constant even as data scale increases by orders of magnitude.

**Codes of Searching Algorithms**

Binary

```cpp
#include <iostream>
#include <vector>
#include <chrono>

void binarySearch(const std::vector<int>& arr, int target) {
    int low = 0;
    int high = static_cast<int>(arr.size()) - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (arr[mid] == target) {
            return;
        } else if (target < arr[mid]) {
            high = mid - 1;
        } else {
            low = mid + 1;
        }
    }
}

void testBinarySearch(int n) {
    std::vector<int> worstArr(n);
    for (int i = 0; i < n; i++) {
        worstArr[i] = i;
    }
    // trigger worst-case traversal (log2(N) + 1)
    int target = -1;

    auto startWorst = std::chrono::high_resolution_clock::now();
    binarySearch(worstArr, target);
    auto endWorst = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> worstTime = endWorst - startWorst;

    std::cout << "N = " << n << "\n";
    std::cout << "Worst Case: " << worstTime.count() << " ms\n";
}
```

```cpp
int main() {
    std::cout << "Linear Search\n";
    testLinearSearch(1000);
    testLinearSearch(10000);
    testLinearSearch(100000);
    return 0;
}
```

Linear

```cpp
#include <iostream>
#include <vector>
#include <chrono>

void linearSearch(const std::vector<int>& arr, int target) {
    for (size_t i = 0; i < arr.size(); i++) {
        if (arr[i] == target) {
            return;
        }
    }
}

void testLinearSearch(int n) {
    std::vector<int> worstArr(n);
    for (int i = 0; i < n; i++) {
        worstArr[i] = i;
    }
    // trigger worst-case traversal (N)
    int target = -1;

    auto startWorst = std::chrono::high_resolution_clock::now();
    linearSearch(worstArr, target);
    auto endWorst = std::chrono::high_resolution_clock::now();
    std::chrono::duration<double, std::milli> worstTime = endWorst - startWorst;

    std::cout << "N = " << n << "\n";
    std::cout << "Worst Case: " << worstTime.count() << " ms\n";
}

int main() {
    std::cout << "Linear Search\n";
    testLinearSearch(1000);
    testLinearSearch(10000);
    testLinearSearch(100000);
    return 0;
}
```

```cpp
int main() {
    std::cout << "Binary Search\n";
    testBinarySearch(1000);
    testBinarySearch(10000);
    testBinarySearch(100000);
    return 0;
}
```

**Conclusion**
(LARA)
This Hands-on activity has introduced the concept of algorithmic complexity where it involves measuring the

time and space required by an algorithm based on the given input size. It illustrates how the theoretical analyses of sorting and searching algorithms determine the experimental performance, which shows how efficient the algorithm is based on the time it takes to sort or search through a given data input. The activities compare two sorting and searching algorithms based on their time and space complexity for theoretical and experimental outputs, which show their key differences for quadratic and logarithmic time complexities. By demonstrating these concepts in experimental testing, the algorithms can be tested with real hardware, and reveal certain flaws and errors in each implementation of the provided algorithms.

**(**MENCIDOR**)**
This activity demonstrated how theoretical time complexity directly dictates practical software performance, illustrating that algorithmic efficiency, rather than hardware speed, is the primary determinant of system scalability as input size n increases. Through comparing sorting (Bubble Sort and Merge Sort, also searching Linear Search and Binary Search) techniques, the procedure and supplementary analysis highlighted critical engineering trade offs. While lower complexity algorithms scale remarkably well over large datasets, they often require structural prerequisites like pre sorted data. Overall, we successfully connected mathematical bounds with empirical benchmarking, with our main area for personal improvement being the mastery of high precision profiling tools to measure microsecond level execution differences across various compiler optimization levels.

## 3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

**Tools Used:** **Gemini**

*Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.*

**Purpose of Use:** **Grammar Correction, Explanation of Concepts**

*Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.*

**Extent of Use:** **The AI was used to clarify certain concepts that were harder to understand, and correct grammatical errors in writing the analyses.**

*Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.*

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.